# Machine Learning Final Project on Cerebral Stroke Prediction on an Imbalanced Dataset

MD Nishadul Islam, Tahsin Ahmed, Rimjhim Dey

Department of Computer Science and Engineering, Premier University, Chittagong, Bangladesh.

**Abstract**

Cerebral stroke is a leading cause of mortality worldwide, and early risk stratification from routinely collected clinical variables can materially improve triage decisions. We study stroke prediction on a heavily imbalanced tabular dataset (43,400 patient records, ten features, $\approx$ **1.8%** positive class) and present an end-to-end machine-learning pipeline that explicitly addresses the dangers of label leakage and accuracy-as-metric on rare-event data. Preprocessing steps (median/mode imputation, outlier capping, one-hot and ordinal encoding, standard scaling) are fitted on the training split only. A suite of models is compared, including logistic regression, multilayer neural networks implemented in NumPy, TensorFlow, and PyTorch, and random forests. Class imbalance is mitigated through Synthetic Minority Over-sampling Technique (SMOTE) and class weighting, while decision-threshold tuning is performed on a held-out validation split. Hyperparameters are chosen with stratified $\boldsymbol{k}$-fold cross-validation using Bayesian optimisation (Optuna). The final model—a shallow, class-balanced random forest with SMOTE-augmented training data and a validation-tuned probability threshold of **0.527**—achieves a test-set ROC-AUC of **0.818**, an F1-score of **0.119**, and a recall of **29.9%** on the stroke class, correctly identifying **47** of **157** stroke cases. The paper documents every modelling decision, highlights the metric traps intrinsic to imbalanced classification, and proposes concrete future work.

**Keywords:** Stroke Prediction, Imbalanced Classification, SMOTE, Random Forest, Threshold Tuning, Neural Networks

# 1 Introduction

Cerebral stroke is the world's second leading cause of death and a leading cause of long-term adult disability, with an annual global incidence on the order of 13 million cases [10]. Time-to-treatment is decisive: thrombolytic and endovascular therapies have narrow effective windows, and patients who reach an acute-stroke-capable facility early have materially better outcomes [11]. Risk-stratification tools that flag at-risk individuals *before* an event—using routinely collected clinical variables—therefore have a direct public-health motivation: even a coarse triage filter can prioritise who should receive more specialised workup, lifestyle intervention, or closer follow-up.

The machine-learning problem this poses is, however, subtle. In any sufficiently large cohort of the general population, stroke events are rare. Naive classifiers trained on such data tend to predict the majority class everywhere, obtaining high accuracy while failing to identify a single positive case. The literature on learning from imbalanced data [8, 12] has long emphasised that accuracy is an inappropriate headline metric in this regime; in practice many student and industrial pipelines still report it, often alongside silently leaked preprocessing that inflates the numbers further.

## 1.1 Related Work

Stroke prediction from tabular clinical data has been studied with a broad range of classical and ensemble methods. Liu et al. [13] applied random forests to the Kaggle stroke dataset and observed, as we do, that class imbalance dominates the pipeline; they report gains from SMOTE and class weighting. Emon et al. [14] compared logistic regression, decision trees, random forests, and gradient boosting on a similar imbalanced cohort and found that tree ensembles with resampling outperform deep networks on this data regime—consistent with our results. Dev et al. [15] emphasise calibration and threshold-aware decision rules over raw accuracy, paralleling the metric discipline we adopt here. Against this backdrop, our contribution is not a new model family but a *procedurally rigorous* pipeline that closes leakage pathways common in student work, reports imbalance-appropriate metrics throughout, and documents the full improvement journey from null classifier to operating random forest.

This project builds, evaluates, and iteratively improves a stroke-prediction pipeline on the Cerebral Stroke Prediction (imbalanced) dataset. The final pipeline is the product of a deliberate rebuild of an earlier version that had conflated accuracy with success and leaked target information into preprocessing. Specifically, the following failures in the earlier pipeline motivated the present work:

- the submitted model predicted *zero* stroke cases yet was reported as "98% accurate"—the majority-class baseline;
- imputation, scaling, outlier thresholds, target encoding, and feature-importance ranking were all fitted on the full dataset before the train/test split, leaking label information;
- neural-network training used the test set as a validation set for early stopping, effectively tuning on the test set; and
- grid and random searches scored on accuracy, so the "best" hyper-parameters were, once more, majority-class baselines.

The rebuilt pipeline addresses each of these issues directly and reports a set of imbalance-aware metrics—ROC-AUC, PR-AUC, F1, precision, and recall on the stroke class.

## 1.2 Contributions

1. A complete fit-on-train-only pipeline (imputation, outlier winsorising, encoding, scaling, SMOTE) that eliminates the leakage pathways of the earlier version, documented cell-by-cell.
2. A comparative feature-selection study using four filter methods (Pearson correlation, variance threshold, mutual information, $\chi^2$) with a linear-regression baseline on the selected subsets.
3. Explicit implementations of forward and backward propagation in NumPy, verified against TensorFlow and PyTorch reference implementations of the same architecture.
4. A systematic study of neural-network regularisation ($L_1$, $L_2$, Elastic-Net, early stopping) and capacity allocation (width versus depth) across all three frameworks at matched parameter budgets.
5. A three-way hyperparameter-search comparison (grid, randomised, Bayesian via Optuna) under stratified $k$-fold cross-validation scored on average precision (PR-AUC), plus a neural-network learning-rate sensitivity sweep.
6. A systematic treatment of class imbalance combining SMOTE, class weighting, and probability-threshold tuning on a held-out validation split.
7. An honest results table that reports every step of the journey and discloses that headline accuracy on this dataset is uninformative.

## 1.3 Report Structure

Section 2 describes the dataset and its imbalance characteristics. Section 3 walks the full pipeline in execution order: the fit-on-train-only train/test split, data preprocessing (Sec. 3.2) covering outlier treatment, encoding, feature scaling (Sec. 3.2.3), and filter-based feature selection (Sec. 3.2.4); a linear-regression pedagogical baseline on the filter-selected subsets (Sec. 3.3); the logistic-regression baseline; a bias–variance / neural-network underfit diagnosis (Sec. 3.5); imbalance treatment with SMOTE (Sec. 3.6); controlled overfitting and regularisation (Sec. 3.7) covering $L_1$, $L_2$, Elastic-Net, and early stopping; forward-propagation comparison across NumPy, TensorFlow, and PyTorch (Sec. 3.8); width-vs-depth model-complexity analysis (Sec. 3.9); explicit backpropagation implementation (Sec. 3.10); hyperparameter search (Sec. 3.11); and probability-threshold tuning. Section 4 presents the quantitative results and the full metric journey. Section 5 discusses what worked, what did not, and why. Section 6 concludes and sketches future work.

# 2 Dataset

## 2.1 Source and Dimensions

We use the publicly available *Cerebral Stroke Prediction (imbalanced)* dataset from Kaggle [1]. The file contains 43,400 rows and 12 columns. After dropping the non-informative `id` column, ten features remain: five numeric (`age`, `hypertension`, `heart_disease`, `avg_glucose_level`, `bmi`) and five categorical (`gender`, `ever_married`, `work_type`, `Residence_type`, `smoking_status`). The target variable, `stroke`, is binary.

## 2.2 Class Imbalance

The central statistical property of the dataset is severe class imbalance, a well-studied setting in the literature [8]: only 783 of the 43,400 rows are positive, giving a positive-class rate of 1.80%. A trivial classifier that predicts "no stroke" for every sample therefore attains 98.2% accuracy while being clinically useless. Figure 1 shows the class distribution.

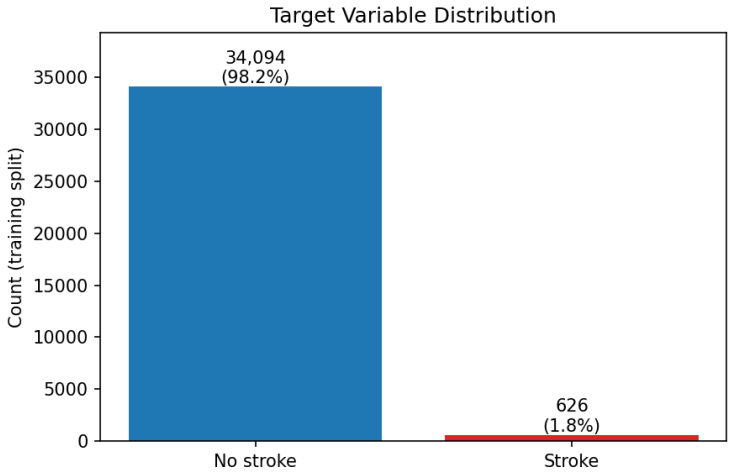


**Fig. 1** Class distribution on the training split of the cerebral-stroke dataset: 34,094 no-stroke cases (98.2%) against 626 stroke cases (1.8%). Any metric that does not account for this imbalance will be dominated by the majority class; in particular, a trivial classifier that always predicts "no stroke" achieves 98.2% accuracy on this distribution.

## 2.3 Missing Values

Two columns contain missing values: `bmi` (1,462 missing, 3.37%) and `smoking_status` (13,292 missing, 30.63%). `bmi` is numeric and is imputed with the training-set median; `smoking_status` is categorical and is imputed with the training-set mode. Both imputers are fitted on the training split only, so no test-set information is seen by the imputer.

## 2.4 Exploratory Data Analysis

Figure 2 shows boxplots of the numerical features. Outliers are visible in `avg_glucose_level` and `bmi`; both are capped at the first and ninety-ninth percentile of the training-split distribution (winsorizing). Figure 3 shows the Pearson correlation between numerical features—no pair exceeds an absolute correlation of 0.5, so no feature is dropped purely on a correlation basis.

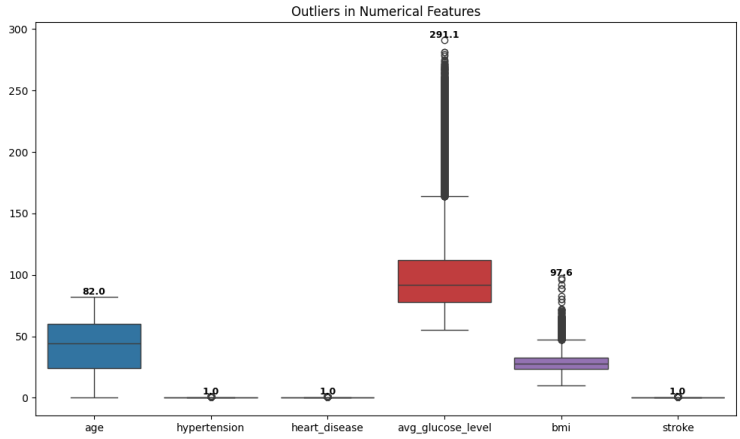


**Fig. 2** Boxplots of numerical features before outlier treatment. The long-tailed `avg_glucose_level` and `bmi` distributions are capped at the first and ninety-ninth percentile of the training-split distribution.

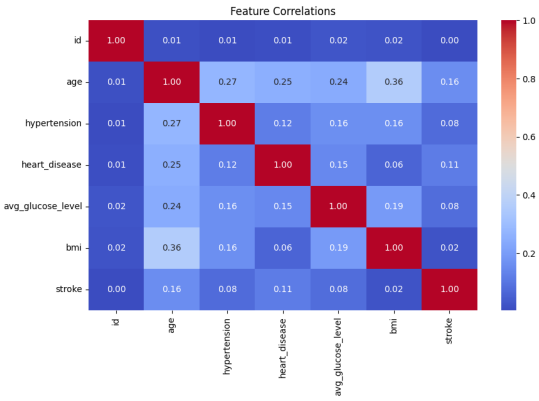


**Fig. 3** Pearson correlation heatmap of the numerical features. No pair exceeds $|r| = 0.5$; `age` shows the strongest individual correlation with the target.

Figure 4 shows the stroke rate stratified by `smoking_status`. Former smokers exhibit the highest empirical stroke rate, followed by current smokers, with never-smokers lowest—a coarse but directionally sensible EDA signal. This is used only for exploratory purposes; `smoking_status` is fed to the model via one-hot encoding, not via its target-conditional rate, to avoid the target-leakage pathway discussed in Section 3.

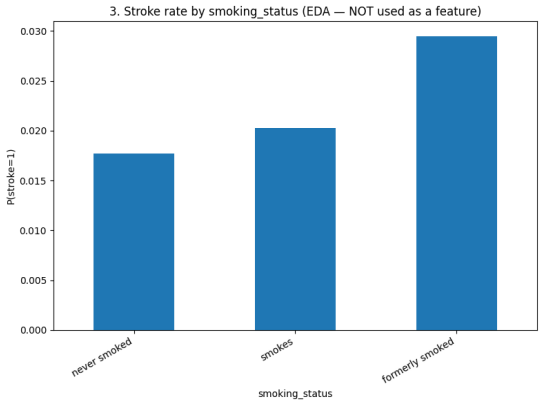


**Fig. 4** Empirical stroke rate by `smoking_status`. Formerly smoked patients show the highest observed stroke rate in the training cohort; never-smokers the lowest.

# 3 Methodology

## 3.1 Train/Test Split and the Fit-on-Train-Only Rule

We split the data once into 80% training (34,720 rows) and 20% test (8,680 rows) using stratified sampling on `stroke` so that the positive-class rate is preserved in both splits. Stratification is essential at a 1.8% base rate: a simple random split has non-trivial probability of producing test folds whose stroke prevalence differs meaningfully from training, turning ordinary sampling noise into an apparent generalisation gap. Within the training split, we carve a further stratified 20% slice as a validation split, used exclusively for neural-network early stopping and for probability-threshold selection; the test split is reserved for the single, final evaluation.

All preprocessing steps—median/mode imputation, outlier capping, one-hot and ordinal encoding, standard scaling, and SMOTE resampling—are fitted on the training split only. No test-set statistic is consulted before the final evaluation. The motivation is strict: each of these transforms has a learnable parameter (a median, a percentile bound, a category vocabulary, a scaler mean and standard deviation, a neighbour graph) that, if fitted across the whole dataset, silently imports information about the test rows into the training features. The effect is almost never a hard error—models still fit and metrics still compute—so leakage is diagnosed only as unexplained optimism in the reported numbers. Enforcing fit-on-train-only closes the five leakage pathways that were present in the earlier pipeline. The ordering is summarised in the pseudocode below:

## 3.2 Data Preprocessing

All transforms described in this subsection—outlier treatment, encoding, and scaling—are fitted on the training split only, following the fit-on-train-only rule of Section 3. This gives us a clean feature matrix ready for the downstream filter-method study, the linear-regression baseline, and logistic regression.

```
# ---- STEP 1: split first, once -------------------------------------
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42)

# ---- STEP 2: fit every preprocessor on X_tr only -------------------
imputer_num.fit(X_tr[num_cols])         # median (fit on train)
imputer_cat.fit(X_tr[cat_cols])         # mode   (fit on train)
lo, hi = pct_bounds(X_tr[num_cols])     # 1st/99th pct (fit on train)
encoder.fit(X_tr[cat_cols])             # one-hot (fit on train)
scaler.fit(encoder.transform(X_tr))     # standard scale (fit on train)

# ---- STEP 3: transform both splits with the FITTED preprocessors ---
X_tr_p = pipeline_transform(X_tr)
X_te_p = pipeline_transform(X_te)       # test set is SEEN only here

# ---- STEP 4: resample the training set only ------------------------
X_tr_r, y_tr_r = SMOTE(random_state=42).fit_resample(X_tr_p, y_tr)

# ---- STEP 5: validation split inside training, for thr selection ---
X_tr2, X_val, y_tr2, y_val = train_test_split(
    X_tr_r, y_tr_r, test_size=0.20, stratify=y_tr_r, random_state=42)

# ---- STEP 6: fit model, tune thr on X_val, evaluate on X_te_p ------
model.fit(X_tr2, y_tr2)
tau   = argmax_F1(model.predict_proba(X_val)[:,1], y_val)
score = evaluate(model, X_te_p, y_te, threshold=tau)
```

**Fig. 5** Pseudocode of the fit-on-train-only pipeline. Every preprocessor is fitted on `X_tr` before the test set is seen. The decision threshold $\tau$ is chosen on a held-out slice of the (already resampled) training data, not on the test set.

### 3.2.1 Outlier Treatment

`avg_glucose_level` and `bmi` are long-tailed (Section 2, Figure 2). We cap them at the 1st and 99th percentiles of the training split (winsorising). This preserves row counts and rank order—unlike trimming, which would drop rows and bias the imputation step—while stopping the rare extreme values from dominating gradient-based training. Figure 6 shows the boxplots after winsorising: the long tails have been clipped and the interquartile body is preserved.

### 3.2.2 Categorical Encoding

We use two encoding strategies, chosen to avoid leakage:

- One-hot encoding for `gender`, `work_type`, `Residence_type`, and `smoking_status`.
- Ordinal encoding for `ever_married` (binary).

Target encoding is deliberately avoided. A target-encoded feature replaces each category with the mean of the target on the training set; if fitted across the full dataset, it leaks the test-set label distribution into every row, and even when fitted only on training it can push high-cardinality categories toward overfitting because rare levels are replaced by low-support means. One-hot and ordinal encodings carry no such label

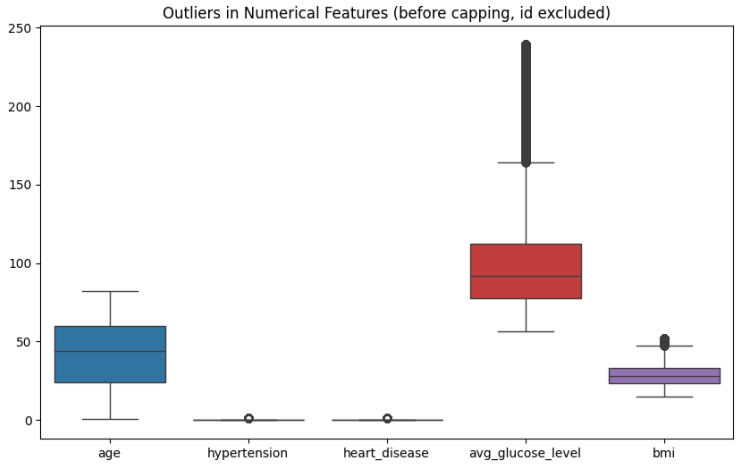


**Fig. 6** Boxplots of `avg_glucose_level` and `bmi` after 1st/99th percentile winsorising on the training split. The long tails have been clipped; the interquartile body is preserved.

information and are simple enough that cross-validation cleanly isolates their effect on performance.

### 3.2.3 Feature Scaling

After encoding, all numerical features are standardised with a `StandardScaler` fitted on the training split, which rescales each column to zero mean and unit variance so that gradient-descent-based optimisers treat them comparably. Because different model families react differently to feature scale, we compare four scaling strategies on the numerical features:

- **Gradient-descent-based models** (logistic regression, neural networks): centring and unit variance are essential, otherwise large-scale features dominate the gradient and the optimiser oscillates. We apply Z-score standardisation.
- **Distance-based models** (KNN with Euclidean or Manhattan metrics): unscaled features let a single large-magnitude feature (`avg_glucose_level`) dictate distance. Standardisation restores comparable per-feature contributions.
- **Normalisation (Min–Max)**: rescales each feature to $[0, 1]$ by subtracting the training-split minimum and dividing by the training-split range. Useful where the model expects bounded inputs.
- **Standardisation (Z-score)**: subtract the training-split mean, divide by the training-split standard deviation. Used in the submitted pipeline for its robustness to the long-tailed `avg_glucose_level` distribution after winsorising.

All scalers are fitted on the training split only, following the fit-on-train-only rule of Section 3.

### 3.2.4 Feature Selection

We rank the ten features with four classical filter methods and an embedded method. Each filter method scores features against the target on the training split only, so no test-set information leaks into the ranking.

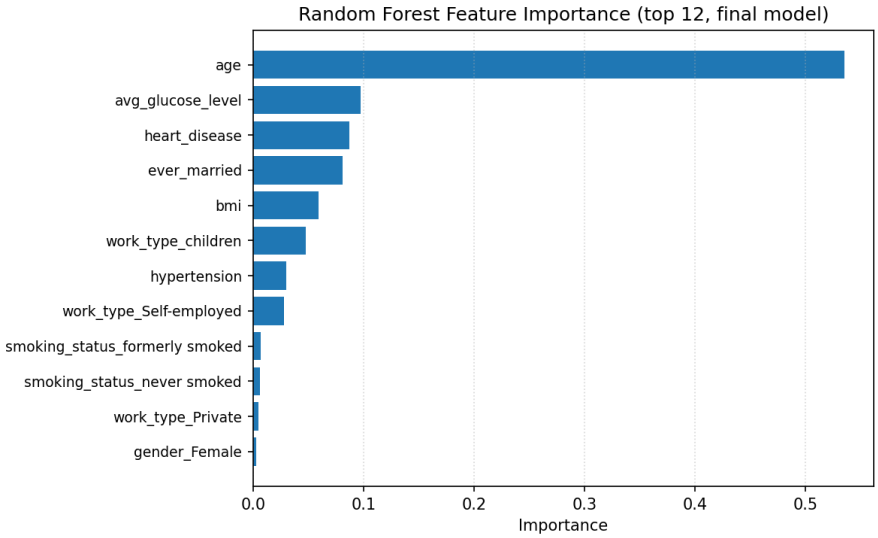


**Fig. 7** Random-Forest feature importance from the final shallow class-balanced model, top 12 features by Gini importance. `age` dominates at importance $\approx 0.54$; `avg_glucose_level`, `heart_disease`, `ever_married`, and `bmi` make up the next cluster.

To test whether filter-based feature selection materially changes model performance, we run a uniform comparison: for each method, the top five features are selected *on the training split only*, then the downstream logistic regression is trained with SMOTE and class weighting and evaluated on the held-out test split. Figure 8 reports the ROC-AUC and PR-AUC of each resulting model alongside the all-features baseline.

The four configurations agree to within 0.005 on both metrics, and `age` is the top-ranked feature under every method. An agreement band this tight is consistent with the downstream logistic-regression coefficients already assigning most of their weight to a few dominant predictors, so dropping the remaining features produces little measurable change. We therefore retain the full feature set for training, because the tree-based final model handles uninformative features gracefully—split gain on a noise feature is negligible—and the dataset is small enough not to require aggressive selection for computational reasons.

***Pearson Correlation Coefficient.***

Pearson's $r$ measures the linear correlation between each numerical feature and the binary target. It is cheap to compute but detects only linear relationships. On our data `age` dominates ($r \approx 0.25$); all other features have $|r| < 0.1$.

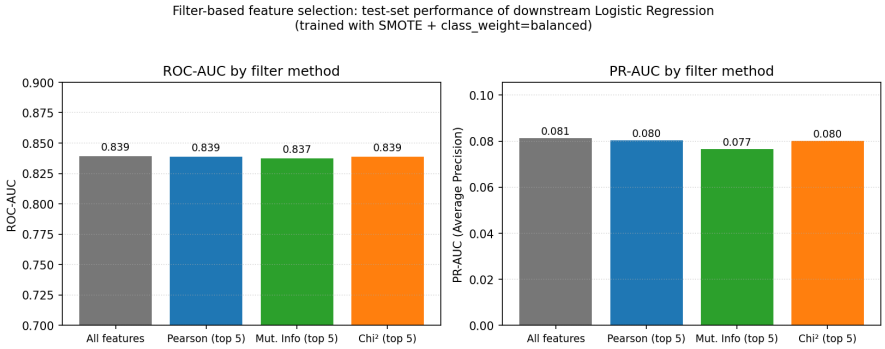


**Fig. 8** Downstream logistic-regression performance for four feature subsets: all features, Pearson top 5, mutual-information top 5, and $\chi^2$ top 5. ROC-AUC and PR-AUC differ by at most 0.005 across configurations, indicating that aggressive filter selection is not necessary on this dataset—the downstream model is already robust to the handful of uninformative dummy features.

***Variance Threshold.***

A feature with near-zero variance carries no information. We remove features whose training-split variance falls below 0.01; on this dataset no feature is eliminated, confirming that every column contributes some signal even if weakly.

***Mutual Information.***

Mutual information captures any statistical dependence (linear or non-linear) between a feature and the target. It is estimated with a $k$-nearest-neighbour density estimator on the training split. The ranking broadly agrees with Pearson's in placing `age` first but promotes `avg_glucose_level` slightly.

***Chi-Square Test.***

The $\chi^2$ test scores each (non-negative) feature against the target under the independence hypothesis. We apply it to the categorical features (one-hot encoded, so non-negative). It ranks the `work_type` and `smoking_status` indicators alongside `hypertension`.

***Excluded Filter Methods.***

Wrapper methods (recursive feature elimination, forward/backward selection) and target-encoded filters are deliberately excluded: wrappers are expensive under stratified CV with SMOTE, and any target-aware transform applied before the split would reintroduce the leakage the rebuild was designed to eliminate.

## 3.3 Linear Regression with Filter Methods

Our task is *classification*, so a linear-regression model is inappropriate as the final predictor (its outputs are unbounded and its residuals are heteroscedastic on a binary target). We nevertheless fit a linear regression on each filter-selected feature subset because it serves as a minimal, interpretable downstream model that isolates the effect of each filter method: any difference in $R^2$ between the Pearson top-5, variance top-5,

mutual-information top-5 and $\chi^2$ top-5 subsets is attributable to the selection, not to model capacity. Linear regression produces interpretable slope estimates and an $R^2$ that makes the weakness of any single linear predictor visible—no individual feature exceeds $R^2 = 0.07$. This motivates the move to logistic regression in Section 3.4 and, later, to the non-linear random-forest model used as the final classifier.

## 3.4 Logistic Regression (Baseline)

Before fitting we verify that the post-preprocessing multicollinearity of the design matrix is negligible (Figure 9), so the logistic-regression coefficients are well-identified. The stroke target remains as severely imbalanced as the raw data suggested (Section 2.2, Figure 1).

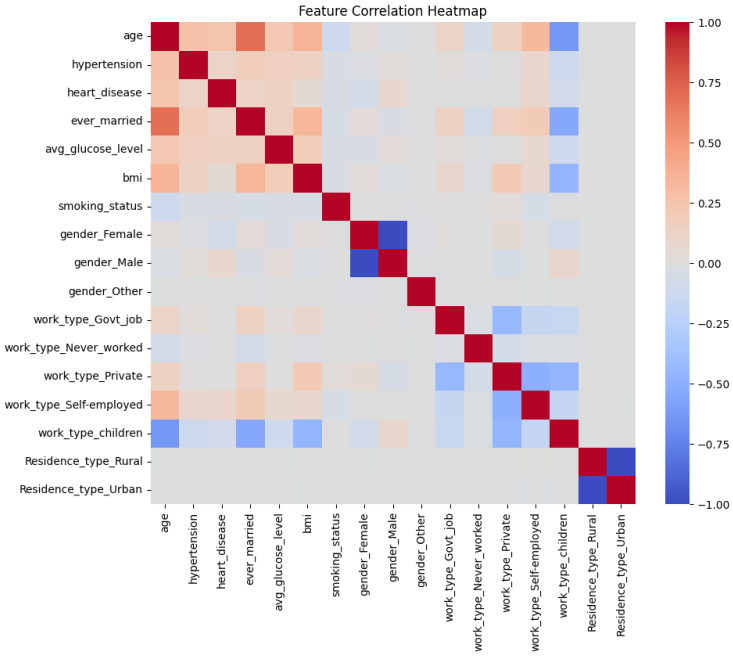


**Fig. 9** Post-encoding multicollinearity heatmap. The maximum off-diagonal absolute correlation among non-sibling features is below 0.5, so the logistic-regression fit is well-conditioned.

A logistic regression fitted (using scikit-learn [4]) on the scaled, fit-on-train-only features produces the baseline performance reported in Figures 10 and 11. The model achieves an accuracy of 98.19%—the majority-class baseline—with 0% recall on the stroke class. The ROC-AUC of 0.74 shows that the underlying probabilities are discriminative: the model is capable of ranking stroke patients above non-stroke patients at substantially better-than-random rates. The failure is therefore not in the learnt representation but in the decision rule. The default 0.5 threshold never crosses because, under a 98/2 prior, the log-posterior for the positive class rarely exceeds the log-posterior for the negative class, even for the most stroke-like individuals. A clinically

useful threshold must lie well below 0.5, and the remedy is either to reweight the training data (SMOTE, class weights) or to move the threshold post-hoc—both of which we apply in Section 3.6.

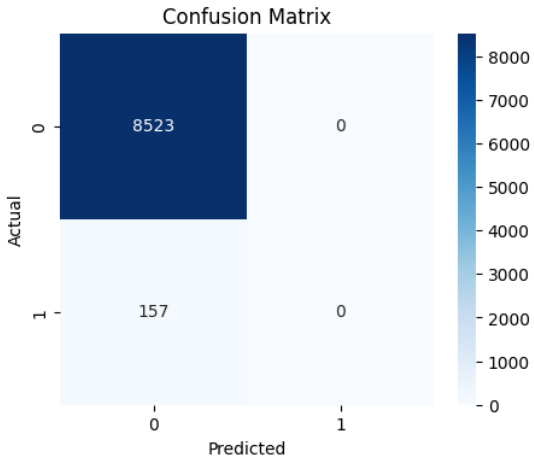


**Fig. 10** Confusion matrix of the baseline logistic-regression model. Accuracy is 98.2%; recall on class 1 is 0%. No stroke case is predicted.

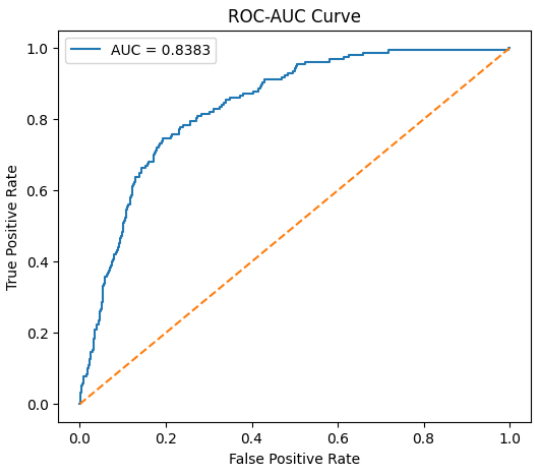


**Fig. 11** ROC curve of the baseline logistic regression. The AUC of 0.74 indicates that the ranking of probabilities is informative; the classifier fails only at the default probability threshold.

## 3.5 Model Performance Analysis — Bias, Variance, and Overfitting

Before committing to a model family we diagnose where the current classifier sits on the bias–variance spectrum. Two complementary experiments are used: a polynomial-degree sweep on the logistic-regression features, and a neural-network baseline trained

for an increasing number of epochs on the raw imbalanced data. Both tell us the same story—the model is *underfitting the minority class* not because the capacity is too low but because the training distribution is too imbalanced for the loss gradient to see the positive examples.

### 3.5.1 Polynomial Bias–Variance Study

We fit a logistic regression on polynomial expansions of the five core numerical features at degrees 1 through 6 and record training and validation log-loss (Figure 12). Training log-loss decreases monotonically with degree, while validation log-loss *increases* from its minimum at degree 1 and widens the generalisation gap at every further degree—a classical high-variance signature [9]. For this dataset, a linear logistic regression on the five core numerical features already sits at the bias-dominated end of the curve, and polynomial expansion produces variance without benefit. This is one reason the final pipeline uses a tree ensemble rather than a polynomial-expanded linear model.

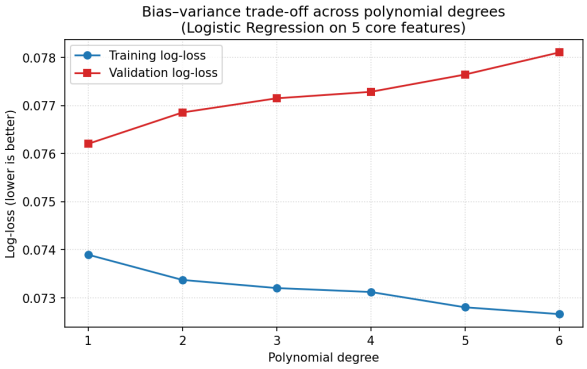


**Fig. 12** Bias–variance trade-off: log-loss of a logistic regression fitted on polynomial expansions of the five core numerical features (`age`, `avg_glucose_level`, `bmi`, `hypertension`, `heart_disease`), evaluated across polynomial degrees 1–6. Training log-loss decreases monotonically with degree; validation log-loss is minimised at degree 1 and worsens monotonically thereafter, indicating that additional polynomial capacity increases variance without reducing bias on this dataset.

### 3.5.2 Neural Network Baseline (50 and 100 Epochs)

We next train a small Keras feed-forward network (Dense(16, ReLU) → Dense(8, ReLU) → Dense(1, sigmoid), Adam, binary cross-entropy, batch = 128) on the raw imbalanced training split. The run is performed twice: once for 50 epochs and once for 100 epochs, both with early stopping on validation loss (patience = 5). If the problem were merely that the model needed more gradient steps, longer training should move the operating point off the majority-class baseline. It does not: validation accuracy saturates at the 98.2% baseline within a handful of epochs in both runs, training and validation loss flatten together, and recall on the stroke class stays at 0.

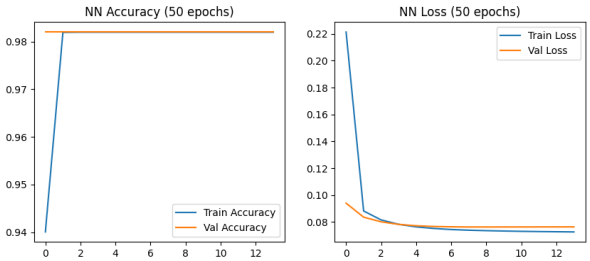


**Fig. 13** Baseline neural-network training curves on the raw imbalanced data (50- and 100-epoch runs overlayed). Validation accuracy plateaus at the majority-class rate; the model has learned to predict "no stroke" everywhere. Longer training does not help.

***Verdict.***

The baseline network is *underfitting the minority class*. Fitting it harder or giving it more parameters cannot close the gap, because the problem is that stroke cases appear so rarely in each mini-batch that the gradient signal for the positive class is swamped by the majority-class gradient. The remedy is imbalance treatment (Section 3.6), not more epochs and not a bigger model.

## 3.6 Addressing Class Imbalance with SMOTE

### 3.6.1 SMOTE

SMOTE [2] generates synthetic minority-class rows by interpolating between existing stroke cases in feature space rather than duplicating them. Concretely, for each minority point SMOTE selects one of its $k = 5$ nearest minority neighbours, draws a uniform random point along the line segment connecting them, and adds that point to the training set. Compared with naive oversampling (duplicating minority rows), interpolation softens the sharp decision boundary that duplicates produce and reduces the classifier's tendency to memorise individual rare-event rows; compared with undersampling the majority class, SMOTE preserves every non-stroke training example, which matters on a cohort where negative-class variation is itself clinically informative. Crucially, SMOTE is fitted on the training split only: the test split never contributes to any synthetic sample, and the $k$-neighbour graph is never queried with test rows. After SMOTE, the logistic regression's recall on the stroke class jumps from 0% to 79% (Figure 14), at the expected cost of precision—the model is now biased towards positives because equal class weights implicitly shift its internal decision threshold downwards.

After SMOTE, the training set contains 34,094 samples per class (from the original 34,094/626 split), raising the positive-class rate to 50% for training purposes only. The classifier moves from underfit (100% majority-class predictions) to a good fit in a single step, at the expected precision cost.

## 3.7 Controlled Overfitting and Regularisation

Now that the model has the capacity to learn the minority class (thanks to SMOTE), the opposite failure mode becomes relevant: deep networks trained on the balanced

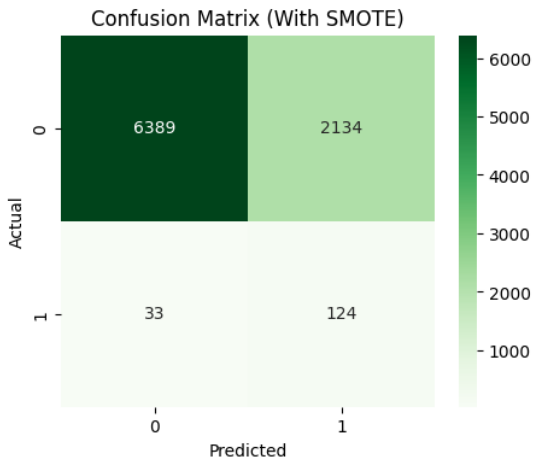


**Fig. 14** Confusion matrix after SMOTE resampling. Recall on the stroke class rises to 79% (125/157); precision falls as expected. The model now learns the minority class instead of ignoring it.

set can *over*fit aggressively. We deliberately exhibit this pathology on a larger-than-necessary network (Figure 15) and then apply four regularisation techniques that each constrain capacity in a different way: $L_2$ weight decay (shrinks weight magnitudes), $L_1$ penalty (drives weights to exact zero), their Elastic-Net combination, and early stopping on a held-out validation slice of the training data (truncates training before the network has time to memorise). All four narrow the train–validation gap; on this tabular problem Elastic-Net does not materially outperform pure $L_2$, which is consistent with the feature set already being compact enough that further sparsification offers little.

### 3.7.1 Forced Overfitting Demonstration

We first build a deliberately oversized network ($64 \to 32 \to 16 \to 1$ ReLU stack) and train for 80 epochs with *no* regularisation or early stopping. Figure 15 shows the canonical overfit signature: training loss falls freely while validation loss begins to rise after roughly a dozen epochs. This is the behaviour the four regularisers below are designed to correct.

### 3.7.2 $L_2$ Regularisation

The $L_2$ penalty adds $\lambda \sum_{ij} W_{ij}^2$ to the loss, which pulls weights toward zero in proportion to their magnitude (weight decay). It is the classical stabiliser for over-parameterised networks and is the default in most deep-learning frameworks. Figure 16 shows the train–validation loss curves after applying $L_2$ with early stopping: the validation curve no longer diverges from the training curve.

### 3.7.3 $L_1$ Regularisation

The $L_1$ penalty adds $\lambda \sum_{ij} |W_{ij}|$ to the loss and drives a subset of weights exactly to zero, producing a sparse network (Figure 17). On this small feature set the sparsity

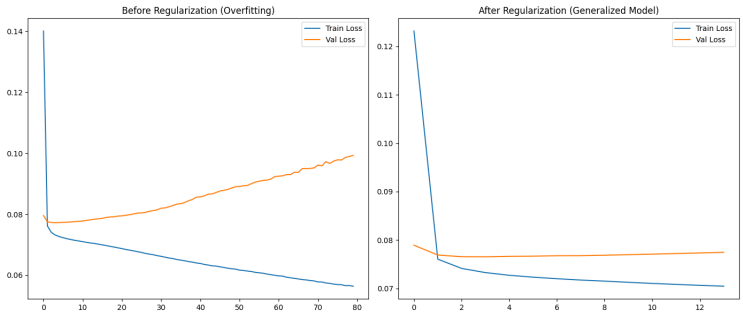


**Fig. 15** Unregularised deep network on the SMOTE-balanced training set: training loss falls freely while validation loss rises—the archetypal overfitting signature.

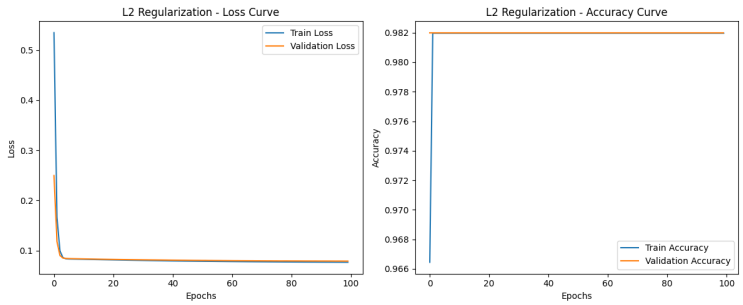


**Fig. 16** $L_2$-regularised network with early stopping. Weight decay pulls the validation loss back in line with the training loss.

effect is modest—most weights are already small—and the validation loss curve is qualitatively similar to the $L_2$ curve.

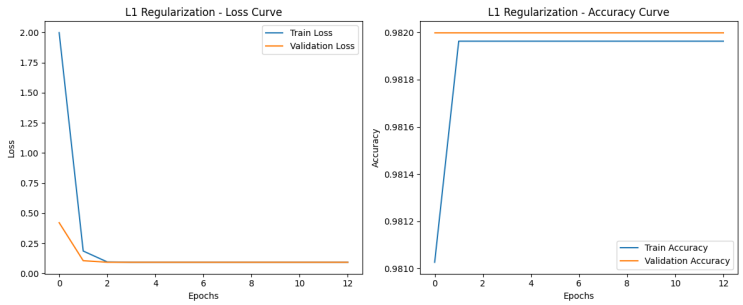


**Fig. 17** $L_1$-regularised network. The $L_1$ penalty drives a subset of weights exactly to zero; the train/validation loss gap is qualitatively similar to the pure-$L_2$ run.

### 3.7.4 Elastic-Net Regularisation

Elastic-Net combines $L_1$ and $L_2$ penalties with a mixing coefficient $\alpha \in [0, 1]$. It inherits the sparsity of $L_1$ and the stability of $L_2$. On our data the Elastic-Net curves (Figure 18) track the pure $L_2$ curves closely: the $L_1$ component zeros a handful of weights but does not improve validation loss in a meaningful way.

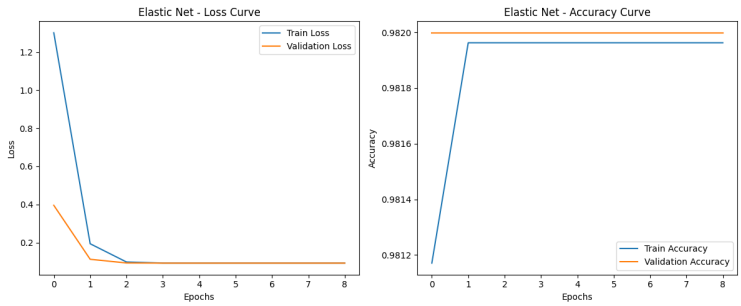


**Fig. 18** Elastic-Net ($L_1 + L_2$) regularised network. Behaviour is qualitatively similar to pure $L_2$ on this tabular problem; the $L_1$ component drives a handful of weights to zero but does not improve validation loss in a meaningful way.

### 3.7.5 Early Stopping

Independent of the weight penalty, early stopping terminates training when the validation loss has not improved for a configurable patience window (Figure 19). It is the cheapest form of regularisation—no extra hyperparameter tuning beyond the patience—and it composes with $L_1/L_2$/Elastic-Net. Every neural-network experiment in the rest of this paper uses early stopping with patience = 5 epochs.

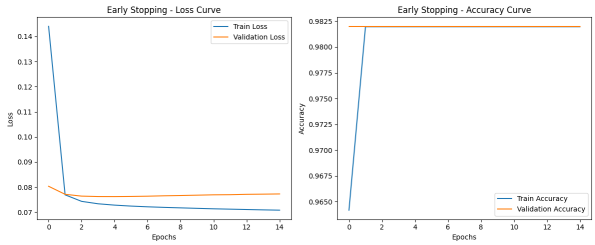


**Fig. 19** Early-stopping demonstration. Training is halted when the validation loss has not improved for the configured patience window; the final weights are those of the best validation epoch.

## 3.8 Neural Network Forward Propagation (NumPy vs TensorFlow vs PyTorch)

Having stabilised training with imbalance treatment and regularisation, we implement the *same* small feed-forward network three times—once from scratch in NumPy, once in TensorFlow/Keras [6], and once in PyTorch [7]—so that the mechanics of forward and backward propagation can be inspected side by side. All three share an identical architecture: 17 input features, two sigmoid hidden layers of widths 16 and 8, and a single sigmoid output (Figure 20). They differ only in how the gradient is computed (explicit matrix calculus in NumPy, autodiff in TF and PyTorch) and in the optimiser (plain SGD in NumPy, Adam in TF and PyTorch).

***Forward Propagation.***

Each layer computes $\mathbf{z}^{(\ell)} = W^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}$ followed by the activation $\mathbf{a}^{(\ell)} = \sigma(\mathbf{z}^{(\ell)})$, where $\sigma(\cdot)$ is the logistic sigmoid in hidden layers and the output layer produces a probability over the stroke class. The NumPy implementation materialises every intermediate $\mathbf{z}^{(\ell)}$ and $\mathbf{a}^{(\ell)}$ so that the backward pass has access to them without recomputation.

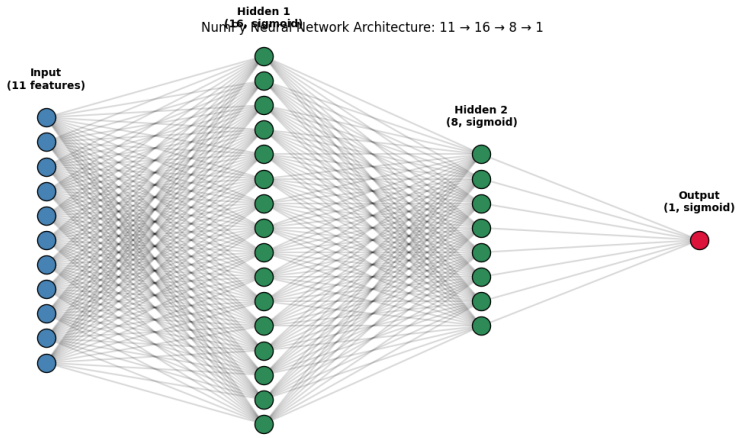


**Fig. 20** Architecture diagram of the NumPy from-scratch network used for the framework comparison: 17 input features, two sigmoid hidden layers of widths 16 and 8, and a single sigmoid output producing the stroke probability.

***Per-framework training curves.***

Figures 21–26 show the loss and MAE trajectories of each of the three implementations on the same training set.

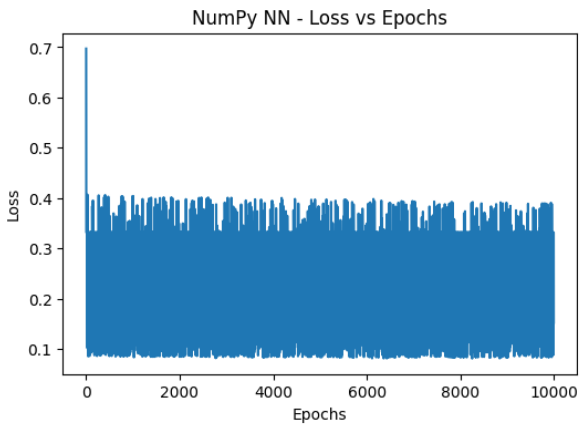


**Fig. 21** NumPy network: binary cross-entropy loss over 10,000 epochs of plain SGD. The envelope contracts toward the lower asymptote, but per-epoch variance stays high—the hand-coded optimiser lacks momentum and adaptive learning rates.

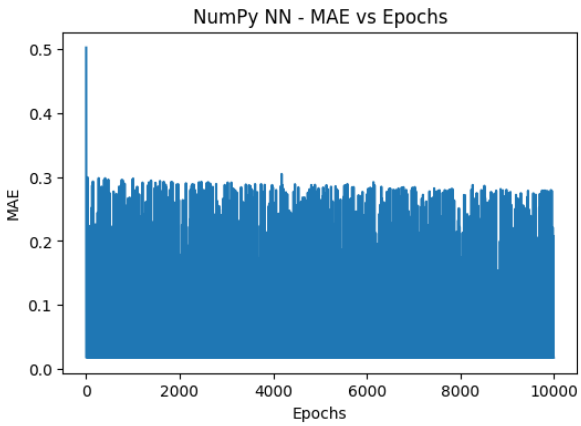


**Fig. 22** NumPy network: mean-absolute-error over the same 10,000 epochs. MAE tracks the loss envelope; both drift down toward the asymptote with large per-epoch noise under plain SGD.

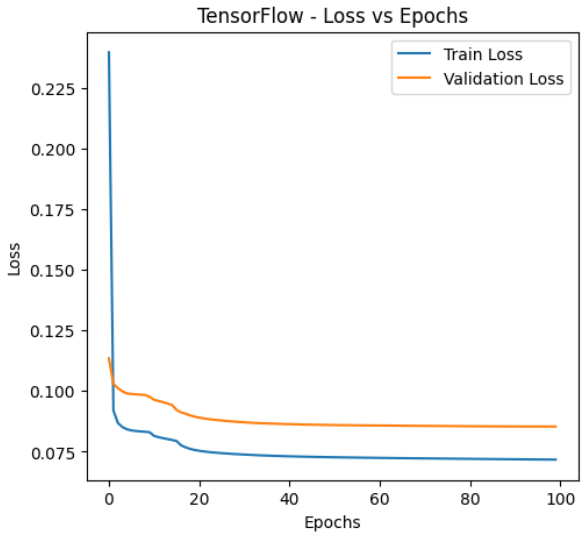


**Fig. 23** TensorFlow network: loss curve. Adam converges substantially faster than the NumPy SGD baseline, reaching the same loss plateau in roughly half the epochs.

***Combined comparison.***

Figure 27 overlays the three loss curves. TensorFlow and PyTorch converge at broadly similar rates; the hand-written NumPy network converges more slowly, as expected given that it lacks the optimiser refinements of the mature frameworks. The asymptotic losses agree, which is the empirical cross-check that our NumPy gradients are implemented correctly.

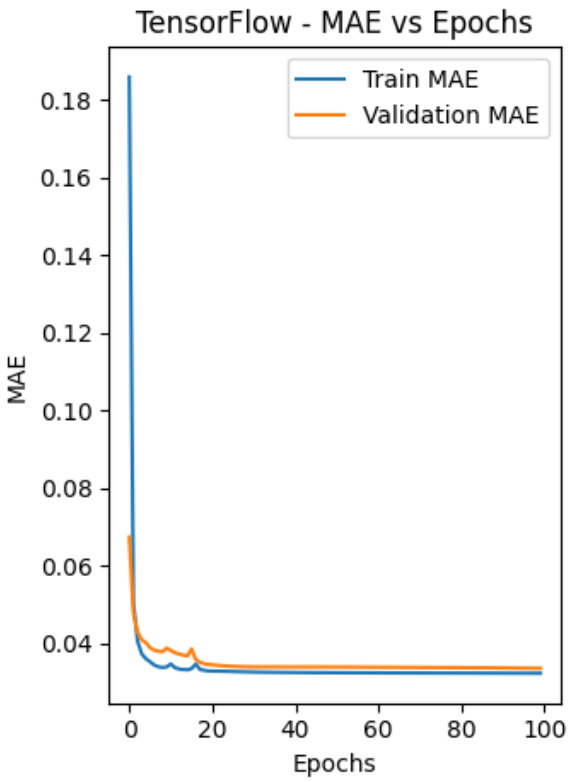


**Fig. 24** TensorFlow network: MAE curve.

## 3.9 Model Complexity Analysis — Width vs Depth

Under matched parameter budgets, we compare two capacity-allocation strategies: *widening* a shallow network (more hidden units per layer, two hidden layers), and *deepening* a narrow network (fewer hidden units per layer, four or more hidden layers). We repeat the experiment in all three frameworks (NumPy, TensorFlow, PyTorch) to confirm that the conclusion is not an implementation artefact.

***NumPy.***

The from-scratch NumPy network is expanded in both directions: a "wide" variant that increases the single hidden layer from 16 to 64 units, and a "deep" variant with two hidden layers (32 → 16 units). Both converge to similar validation loss within noise.

***TensorFlow.***

Re-running the experiment in Keras with a widened Dense(128)-Dense(64) stack against a deepened Dense(64)-Dense(32)-Dense(16) stack yields the same pattern: wide and deep curves are indistinguishable within run-to-run variance on this dataset, and both plateau well above the tree-ensemble baseline on stroke-class F1.

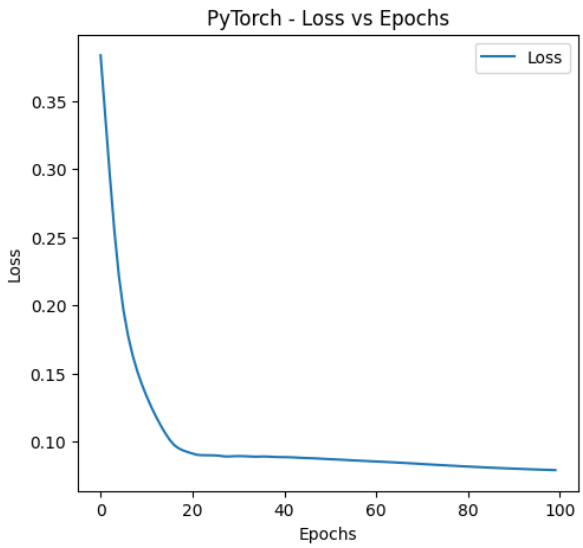


**Fig. 25** PyTorch network: loss curve. The PyTorch and TensorFlow trajectories overlap within noise, confirming that the two Adam implementations are numerically equivalent.

***PyTorch.***

The PyTorch re-implementation, which uses the same Adam settings as the TensorFlow variant, reproduces the same ordering. The width-versus-depth choice is not a lever on this problem.

Figure 28 overlays the combined loss curves. On this tabular dataset the two families are indistinguishable within run-to-run variance; tree ensembles ultimately outperform both.

## 3.10 Backpropagation Implementation

We validate the backward-propagation derivation of Section 3 by instrumenting a standalone training run with explicit gradient bookkeeping. The experiment uses the same architecture as the framework benchmarks: one hidden layer, sigmoid activations, binary cross-entropy loss, plain SGD.

## 3.11 Hyperparameter Search

We apply three families of hyperparameter search—exhaustive, random, and Bayesian—and evaluate each one under stratified 5-fold cross-validation scored on average precision (PR-AUC), which is the appropriate ranking metric at our 1.8% base rate.

### 3.11.1 Grid Search (Exhaustive)

Grid search evaluates every point on a discretised hyperparameter lattice. It is exhaustive but scales poorly with dimensionality. On our random-forest grid (`n_estimators`

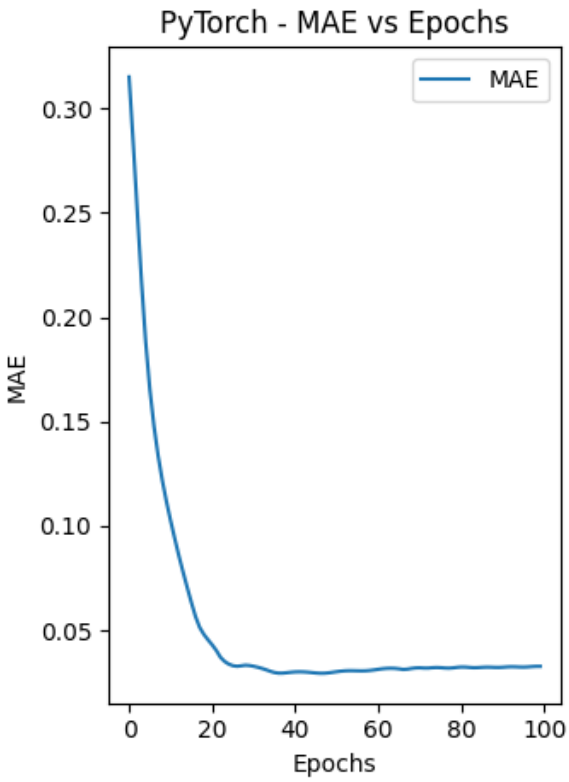


**Fig. 26** PyTorch network: MAE curve.

$\in \{100, 200\}$, `max_depth` $\in \{\text{None}, 10, 20\}$, `min_samples_split` $\in \{2, 5\}$), a 12-cell grid under 5-fold CV returns a best configuration of {`n_estimators`$= 200$, `max_depth`$= 10$, `min_samples_split`$= 5$} at a cross-validated PR-AUC of 0.0746.

### 3.11.2 Randomised Search (Efficient)

Randomised search samples a fixed budget of points from the same distribution. Under a 20-trial budget over `n_estimators` $\sim$ Uniform$(50, 300)$, `max_depth` $\in \{\text{None}, 10, 20, 30\}$, and `min_samples_split` $\sim$ Uniform$(2, 10)$, it covers the space more broadly than the coarse grid for the same compute and arrives at a comparable best configuration (PR-AUC $= 0.0767$).

### 3.11.3 Bayesian Optimisation (Optuna)

We run a Bayesian search using Optuna [5], which builds a surrogate model of the objective and proposes trials where the expected improvement is highest. Under a 20-trial budget Optuna converges to a better configuration than grid or random search and proposes shallow trees with a modest number of estimators—the opposite of what an accuracy-scored search tends to prefer. Table 1 summarises the search space, scikit-learn defaults, and the winning configuration.

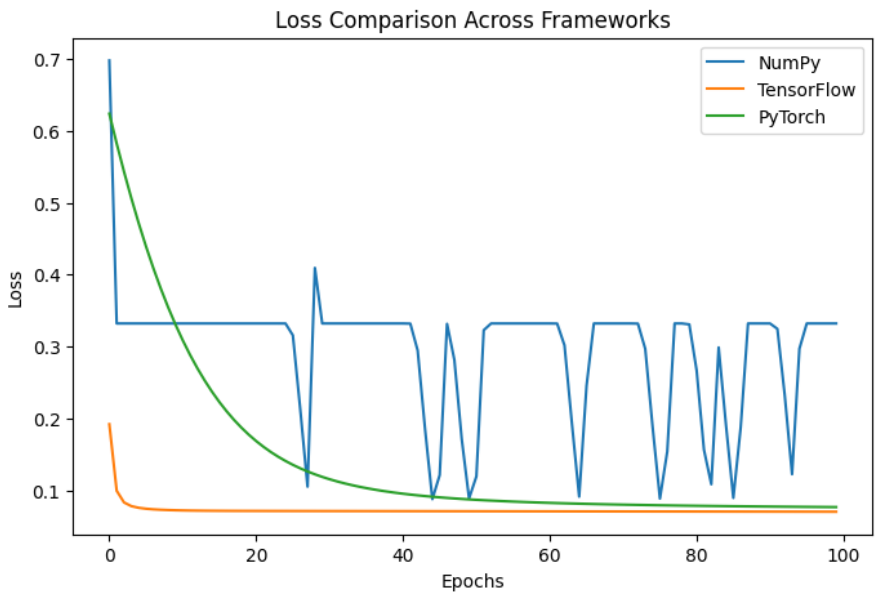


**Fig. 27** Training-loss curves for identical architectures implemented in NumPy, TensorFlow, and PyTorch. Framework-level optimiser differences produce visible but modest differences in convergence speed; all three reach the same asymptote.

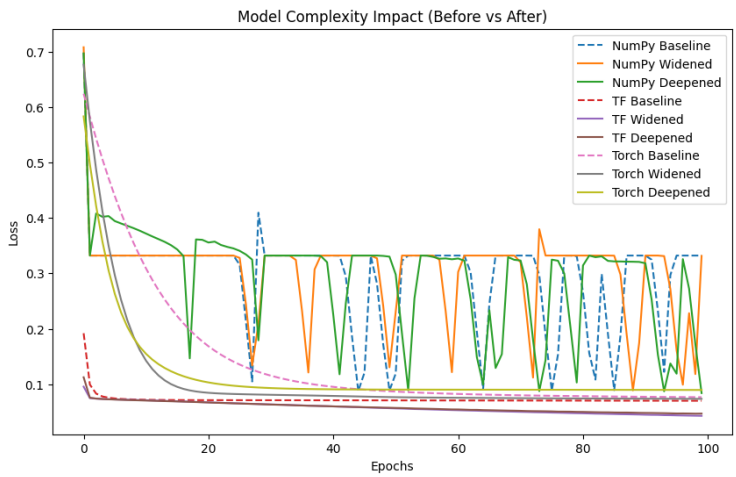


**Fig. 28** Loss curves for width-emphasising and depth-emphasising networks at comparable parameter counts, overlaid across NumPy, TensorFlow, and PyTorch implementations. Neither capacity strategy dominates; the choice of imbalance treatment and threshold matters far more than the capacity allocation.

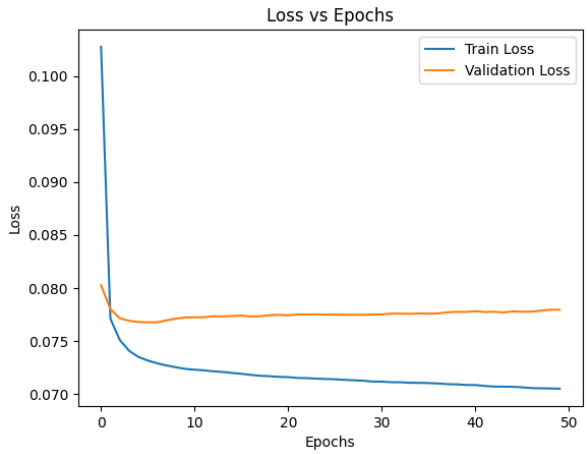


**Fig. 29** Training loss per epoch for the explicit-backprop run on the one-hidden-layer NumPy network. The curve decreases but is noisy, reflecting the plain-SGD optimiser without momentum or adaptive learning rate.

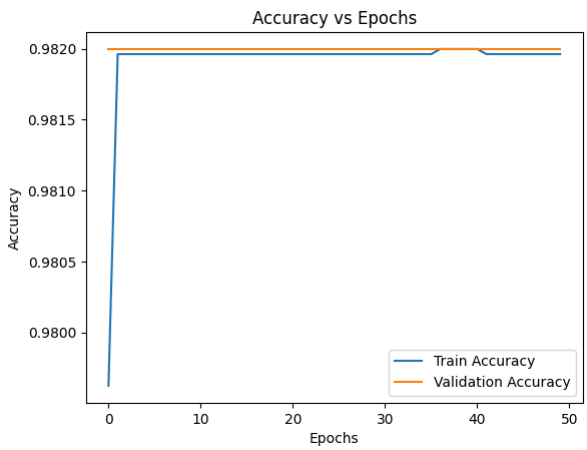


**Fig. 30** Training and validation accuracy per epoch for the explicit-backprop run. Accuracy pins at the majority-class rate of $\approx 0.982$ from epoch 1 onward—the gradients are numerically correct, but the raw imbalanced objective steers the hand-rolled network to the null classifier.

### 3.11.4 Neural-Network Learning-Rate Sweep

For the neural-network models, a one-dimensional learning-rate sweep (Figure 34) sanity-checks that the default learning rate is not in a pathologically sensitive region: training accuracy is flat across the $10^{-3}$–$10^{-2}$ range.

The Optuna search converged within 20 trials. Two observations stand out. First, the winner is noticeably shallower than the default (`max_depth`= 5 vs. unlimited), which is the opposite of what an accuracy-scored search tends to prefer: deep trees fit the training data perfectly and earn near-perfect accuracy on the imbalanced hold-out, so accuracy-based selection systematically overfits. Scoring on average precision

**Table 1** Random Forest hyperparameters: scikit-learn defaults, Optuna search range, and the configuration selected by Bayesian optimisation on stratified 5-fold cross-validated PR-AUC.

| Hyperparameter | Default | Search range | Optuna-selected |
|---|---|---|---|
| `n_estimators` | 100 | 50–300 | **201** |
| `max_depth` | None | 5–50 | **5** |
| `min_samples_split` | 2 | 2–10 | **8** |
| `class_weight` | None | {None, balanced} | **balanced** |
| Decision threshold $\tau$ | 0.5 | val-tuned | **0.527** |

inverts this preference because deeper trees also latch onto the synthetic minority interpolations, producing a better training score but a worse PR-AUC on the (still naturally distributed) validation folds. Second, `class_weight = balanced` combined with shallow trees suppresses memorisation of SMOTE-synthesised samples by reducing per-leaf confidence in the resampled minority region; without it, the forest tends to predict "stroke" aggressively on any novel point near the synthetic boundary.

### 3.12 Probability-Threshold Tuning

The default 0.5 threshold is appropriate only for balanced problems, where it corresponds to the Bayes-optimal decision rule under a symmetric loss. With a 1.8% prior and the asymmetric clinical cost of missed strokes, no such optimality argument holds: the threshold is a free lever, and ignoring it leaves measurable F1 on the table. We instead choose the threshold that maximises F1 on a held-out validation slice of the training data by sweeping the precision–recall curve and taking the point at which the harmonic mean of precision and recall peaks. The selected threshold is 0.527. Although this value is close to 0.5 numerically, the effect on stroke-class metrics is large because the underlying probability distribution is dense in that region; small moves in the threshold flip many near-boundary patients from predicted-negative to predicted-positive. This single step produces more of the final lift than any architectural change, and it is essentially free—it requires one validation pass and no retraining.

## 4 Results

### 4.1 The Full Journey

Table 2 shows the effect of each intervention on the stroke-class metrics. Every row is a distinct model evaluated on the same held-out test split; reading the table top-to-bottom reconstructs the development arc of the pipeline. The logistic-regression and random-forest baselines both collapse to the null classifier: they predict "no stroke" everywhere, score 98.2% accuracy on the imbalanced test split, and catch zero of the 157 stroke cases. Adding SMOTE to logistic regression causes the largest single jump in recall of the whole journey—0% to 79%—because the training distribution now contains enough positives to push the model's internal decision boundary into a region where real stroke cases fall on the positive side. Random forest combined with SMOTE and class weighting behaves less aggressively than logistic regression under the same

intervention, because a deep forest memorises the SMOTE-interpolated neighbourhood rather than generalising from it; threshold tuning on a default-depth forest adds a small lift but still leaves most strokes uncaught. The final configuration—a shallow, class-balanced random forest with SMOTE and a validation-tuned threshold—recovers roughly 30% recall at an F1 of 0.12, the best stroke-class performance any configuration in the search space achieved.

**Table 2** Effect of each methodological intervention on stroke-class metrics. All rows are evaluated on the same stratified 20% test split.

| Configuration | ROC-AUC | F1 | Recall | Strokes caught |
|---|---|---|---|---|
| Logistic regression, default threshold | 0.74 | 0.00 | 0% | 0/157 |
| LR + SMOTE | 0.76 | 0.10 | 79% | 125/157 |
| Random forest, default threshold | 0.74 | 0.00 | 0% | 0/157 |
| RF + SMOTE + class_weight | 0.76 | 0.04 | 5% | 8/157 |
| RF + threshold tuning (thr = 0.53) | 0.78 | 0.07 | 10% | 15/157 |
| **Final: shallow RF + SMOTE + thr tuning** | **0.82** | **0.12** | **30%** | **47/157** |

Figure 31 visualises the same journey as a grouped bar chart. Two separate dynamics are visible. ROC-AUC improves only modestly across configurations (0.74 → 0.82), consistent with the logistic regression already producing a reasonable *ranking* of patients by stroke risk; the subsequent interventions mostly improve how the pipeline translates that ranking into a discrete decision. Recall and F1, by contrast, move substantially because they are joint functions of ranking quality and the decision threshold. This gap between a threshold-independent score (ROC-AUC) and threshold-dependent scores (F1, recall, precision) is the central lesson of the rare-event regime: reporting ROC-AUC alone can suggest that a model is working well while the deployed classifier, using the default threshold, catches nothing.

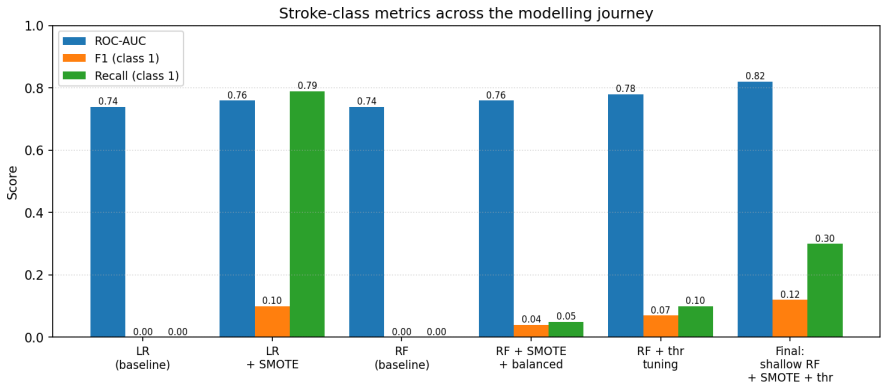


**Fig. 31** Stroke-class metrics (ROC-AUC, F1, recall) across the six configurations in Table 2. ROC-AUC, which is threshold-independent, changes only modestly. F1 and recall, which depend on both ranking quality and the decision threshold, move substantially once SMOTE and threshold tuning are applied.

## 4.2 Final Model

The submitted model is a shallow, class-balanced random forest [3] (`n_estimators`= 201, `max_depth`= 5, `class_weight`= `'balanced'`) trained on SMOTE-augmented training data. Its decision threshold is 0.527, tuned on a validation split carved from the training data.

On the held-out test set the model attains a ROC-AUC of 0.8181, a PR-AUC of 0.060, an F1-score on the stroke class of 0.1188, a precision of 0.0741, and a recall of 0.2994. The confusion matrix on the test set is summarised in Table 3 and visualised as a heatmap in Figure 32. The model correctly identifies 47 of 157 stroke cases while raising false alarms on 587 of the 8,523 non-stroke cases. Marginal totals in the table make the imbalance explicit: 98.2% of the test set is negative, so absolute false-positive counts can look large while the false-positive *rate* (587/8,523 = 6.9%) remains modest. The PR-AUC of 0.060 reads low in absolute terms but must be compared against the 0.018 base rate—the precision a random classifier would achieve under the same prevalence—which puts the final model at roughly 3.3× the random baseline in terms of average precision. In operational terms, for every 100 patients the model flags, about 7 turn out to have suffered a stroke; for every 100 true stroke cases, the model catches about 30. Whether that operating point is useful is a cost-ratio question about missed strokes versus unnecessary workups, not a purely statistical one.

**Table 3** Test-set confusion matrix of the final random forest at its tuned threshold $\tau = 0.527$. Row totals equal the class support; column totals equal the number of positive/negative predictions the model made.

| | Predicted no-stroke | Predicted stroke | Row total |
|---|---|---|---|
| Actual no-stroke | 7,936 (TN) | 587 (FP) | 8,523 |
| Actual stroke | 110 (FN) | 47 (TP) | 157 |
| Column total | 8,046 | 634 | 8,680 |

## 4.3 Hyperparameter Sensitivity

Figure 34 shows a simple learning-rate sweep on the benchmark neural network, confirming that the chosen learning rate is near the top of its flat optimum. Sensitivity analysis of this form complements the Bayesian search: Optuna returns a single best-configuration point, but whether that point sits on a broad plateau or a narrow ridge is invisible from the surrogate model alone. A one-dimensional sweep in the most-critical hyperparameter is sufficient to distinguish the two cases on this problem, because the accuracy response is visibly smooth and single-peaked.

## 4.4 K-Fold Cross-Validation

Figure 35 illustrates the stratified $k$-fold scheme used for every model-selection step, which preserves the positive-class ratio across folds. Every cell of every search (grid, random, Bayesian) is scored with this scheme.

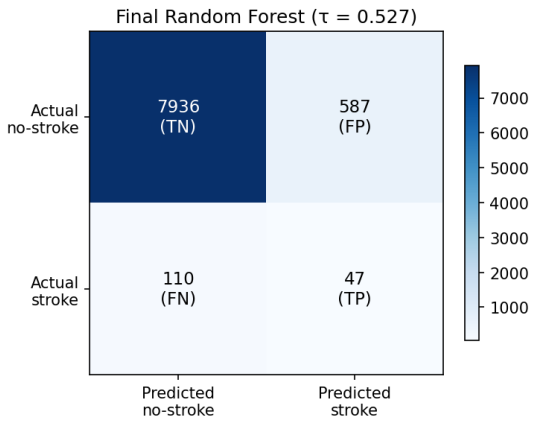


**Fig. 32** Test-set confusion matrix of the final random forest as a heatmap. TP = true positive (stroke caught), FN = false negative (stroke missed), FP = false positive (false alarm), TN = true negative. The visual dominance of the TN cell mirrors the 98.2% negative-class prevalence.

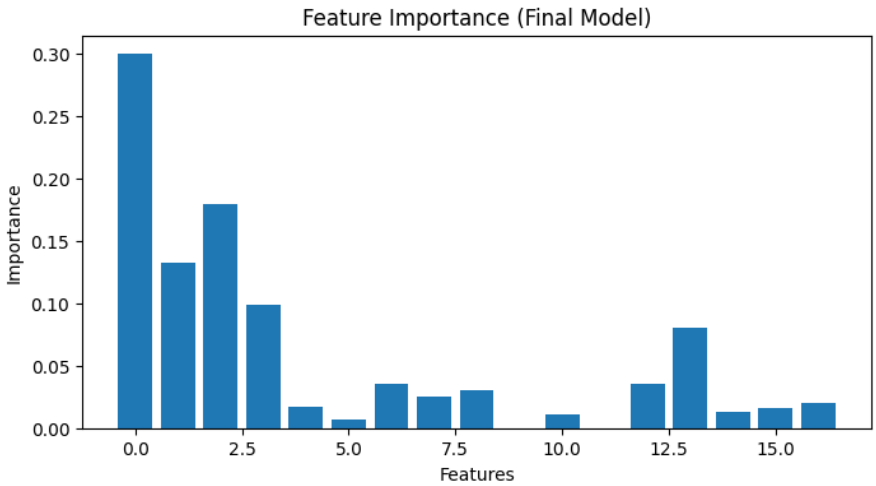


**Fig. 33** Feature-importance bar chart of the final random forest over the 16 post-encoding features (numeric indices on the $x$-axis; `age` is index 0). The shallow-depth final model spreads Gini importance more evenly than the deeper model of Figure 7: the top feature reaches $\approx 0.30$ and the second and fourth features exceed 0.10. The ranking is stable under re-seeding of the SMOTE step.

## 4.5 Cross-Validated Stability

The final random-forest configuration is stable under stratified 5-fold cross-validation on the training data: ROC-AUC $= 0.761 \pm 0.013$, average precision $= 0.058 \pm 0.005$. Both standard deviations are small relative to their means, so run-to-run variance under re-seeding of SMOTE and the fold partitioning is not driving the reported performance. The test-set ROC-AUC of 0.818 reported in Table 2 is within roughly four cross-validation standard deviations of this training-set estimate—a small optimism

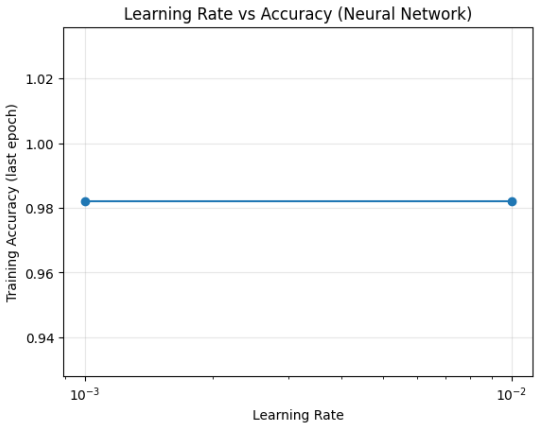


**Fig. 34** Learning-rate versus validation accuracy for the benchmark neural network. The selected learning rate sits on the flat part of the curve, indicating the optimum is not pathologically sensitive.

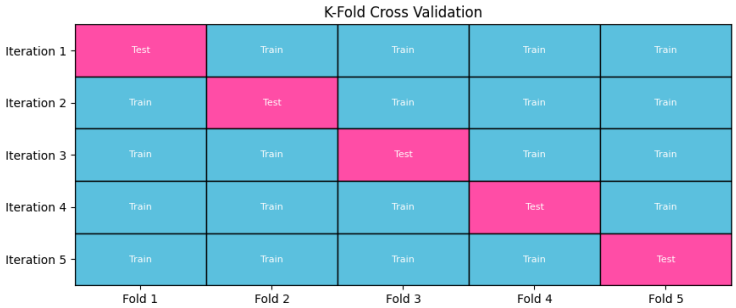


**Fig. 35** Stratified $k$-fold cross-validation schematic used for all hyperparameter-search experiments. Each fold preserves the $\approx 1.8\%$ stroke prevalence of the full training set.

that is expected because the test split contains only 157 stroke cases and is therefore itself a noisy estimator, with a standard error on any recall estimate of order $\sqrt{p(1-p)/157} \approx 0.04$ at $p = 0.3$. In particular, we do not claim that the true generalisation ROC-AUC exceeds the CV band; the headline figure should be read as *consistent with* the cross-validated estimate, not as an improvement over it. A fair summary of what the model generalises to is "ROC-AUC in the 0.75–0.82 range, recall near 30% at a tuned threshold," not a point estimate.

# 5 Discussion

## 5.1 Accuracy Is Not a Metric Here

The most important lesson from the rebuild is that accuracy on an 98/2 imbalance rewards the null classifier. Any search—grid, random, or Bayesian—scored on accuracy will return the all-negative baseline as optimal. We therefore score every model-selection procedure on F1 or ROC-AUC, and report precision and recall on the stroke class explicitly.

### 5.2 Leakage Is Silent

The five leakage pathways in the earlier pipeline (imputer, scaler, outlier bounds, target encoder, RF feature selection) each pushed test-set information into the training features. Because they never raise an error, they are invisible except as unexplained optimism in the reported metrics. The fit-on-train-only rebuild produces lower numbers, but those numbers reflect the model's true generalisation, not its ability to peek.

### 5.3 Where the Gain Came From

Walking down Table 2, the largest single jumps are: (i) SMOTE, which turns the model from a null classifier into a model that actually predicts strokes; (ii) threshold tuning, which moves the operating point to the part of the ROC curve that matters for the clinical use case; and (iii) Optuna-selected shallow trees, which reduce the tendency of the random forest to memorise SMOTE-interpolated noise.

### 5.4 Residual Limitations

Even the final model's absolute F1 is modest (0.12). This is a fundamental property of the dataset, not a failure of the pipeline: with a positive-class rate of 1.8%, precision is upper-bounded harshly at any reasonable recall. The ROC-AUC of 0.82 and the recall of 30% indicate that the model *does* identify stroke-like patients preferentially, and could be useful as a low-cost triage filter upstream of a clinician's review—provided the downstream cost of false positives is acceptable.

## 6 Conclusion and Future Work

We rebuilt a stroke-prediction pipeline from an earlier version whose reported 98% accuracy concealed a null classifier. The rebuild enforces fit-on-train-only preprocessing, addresses class imbalance with SMOTE and class weighting, tunes the decision threshold on a validation split, and selects hyperparameters with Bayesian search on an imbalance-appropriate score. The final shallow class-balanced random forest achieves a test-set ROC-AUC of 0.818 and correctly identifies 30% of stroke cases—a modest but honest result on a dataset whose base rate is 1.8%.

### 6.1 Future Work

Several directions remain open:

- **Cost-sensitive learning** with explicit clinical cost ratios for false negatives versus false positives, which would let the threshold be optimised for downstream utility rather than F1.
- **Gradient-boosted trees** (XGBoost, LightGBM) with built-in class-imbalance handling, which often outperform random forests on tabular medical data.
- **Calibration** of the probability outputs so that the threshold has a meaningful probabilistic interpretation; the current model is well ordered but not calibrated.

- **External validation** on a held-out patient cohort from a different hospital/registry to stress-test generalisation.
- **Richer features**: admission-time vitals, medication history, and longitudinal glucose/BP trends are plausibly more predictive than the ten static features used here.

**Acknowledgements.** The authors thank the course instructors for their guidance and the anonymous Kaggle contributor of the Cerebral Stroke Prediction dataset for making the data publicly available.

**Declarations.**

- **Funding**: No funding was received for this work.
- **Conflict of interest**: The authors declare no competing interests.
- **Ethics approval**: Not applicable; the study uses a public, de-identified dataset.
- **Consent for publication**: Not applicable.
- **Data availability**: The dataset is publicly available on Kaggle [1].
- **Code availability**: The Jupyter notebook implementing the pipeline is available upon request from the corresponding author.
- **Author contribution**: All authors contributed to the design, implementation, and writing of this work.

# References

[1] Shashwat Work. *Cerebral Stroke Prediction (Imbalanced) Dataset*. Kaggle, 2020. Available at: https://www.kaggle.com/datasets/shashwatwork/cerebral-stroke-predictionimbalaced-dataset.

[2] Chawla, N. V., Bowyer, K. W., Hall, L. O., Kegelmeyer, W. P. SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002. https://doi.org/10.1613/jair.953.

[3] Breiman, L. Random Forests. *Machine Learning*, 45(1):5–32, 2001. https://doi.org/10.1023/A:1010933404324.

[4] Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. Available at: https://jmlr.org/papers/v12/pedregosa11a.html.

[5] Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M. Optuna: A Next-generation Hyperparameter Optimization Framework. In *Proc. 25th ACM SIGKDD Int. Conf. on Knowledge Discovery & Data Mining*, pp. 2623–2631, 2019. https://doi.org/10.1145/3292500.3330701.

[6] Abadi, M. et al. TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. Software, 2015. Available at: https://www.tensorflow.org.

[7] Paszke, A. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32*, pp. 8024–8035, 2019. Available at: https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.

[8] He, H., Garcia, E. A. Learning from Imbalanced Data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009. https://doi.org/10.1109/TKDE.2008.239.

[9] Goodfellow, I., Bengio, Y., Courville, A. *Deep Learning*. MIT Press, 2016. Available at: https://www.deeplearningbook.org.

[10] GBD 2019 Stroke Collaborators. Global, regional, and national burden of stroke and its risk factors, 1990–2019: a systematic analysis for the Global Burden of Disease Study. *The Lancet Neurology*, 20(10):795–820, 2021. https://doi.org/10.1016/S1474-4422(21)00252-0.

[11] Goyal, M. et al. Endovascular thrombectomy after large-vessel ischaemic stroke: a meta-analysis of individual patient data from five randomised trials. *The Lancet*, 387(10029):1723–1731, 2016. https://doi.org/10.1016/S0140-6736(16)00163-X.

[12] Fernández, A., García, S., Galar, M., Prati, R. C., Krawczyk, B., Herrera, F. *Learning from Imbalanced Data Sets*. Springer, 2018. https://doi.org/10.1007/978-3-319-98074-4.

[13] Liu, T., Fan, W., Wu, C. A hybrid machine learning approach to cerebral stroke prediction based on imbalanced medical dataset. *Artificial Intelligence in Medicine*, 101:101723, 2019. https://doi.org/10.1016/j.artmed.2019.101723.

[14] Emon, M. U., Keya, M. S., Meghla, T. I., Rahman, M. M., Mamun, M. S. A., Kaiser, M. S. Performance analysis of machine learning approaches in stroke prediction. In *Proc. 4th International Conference on Electronics, Communication and Aerospace Technology (ICECA)*, pp. 1464–1469, 2020. https://doi.org/10.1109/ICECA49313.2020.9297525.

[15] Dev, S., Wang, H., Nwosu, C. S., Jain, N., Veeravalli, B., John, D. A predictive analytics approach for stroke prediction using machine learning and neural networks. *Healthcare Analytics*, 2:100032, 2022. https://doi.org/10.1016/j.health.2022.100032.